Part 2: I audited my own ML project, then took it live

Siddharth Dongardive

Delhi Air Quality Forecasting

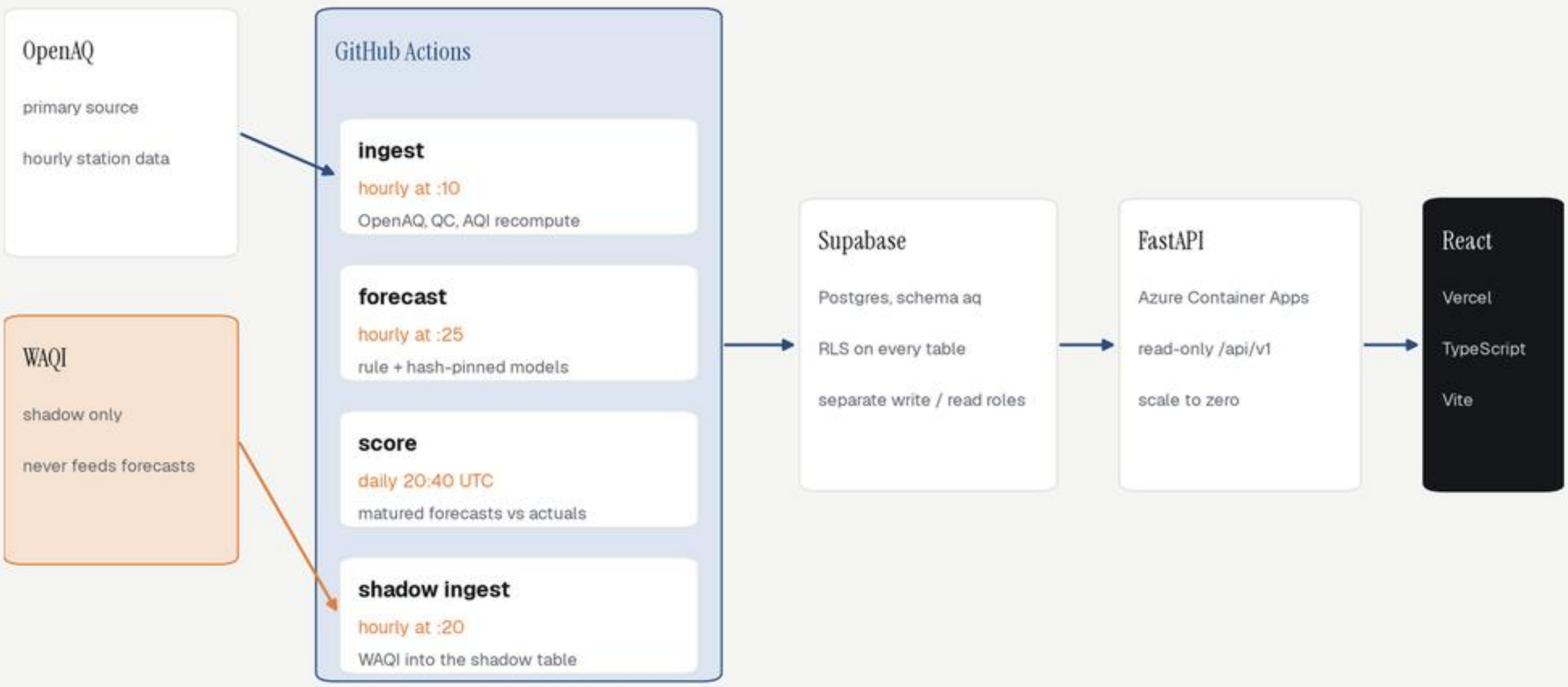
Part 1 vs Part 2: what changed

PART 1		PART 2
1h R^2 0.9958	→	Reported vs persistence (R^2 0.995)
Threshold tuned on test	→	Threshold chosen on validation
Diwali “ban effect”	→	Methods lesson
Streamlit app	→	Live system on Azure + Vercel

From government sensors to a live forecast

How the live system fits together

Scheduled jobs write to one database; a read-only API and the dashboard sit on top. No model server.



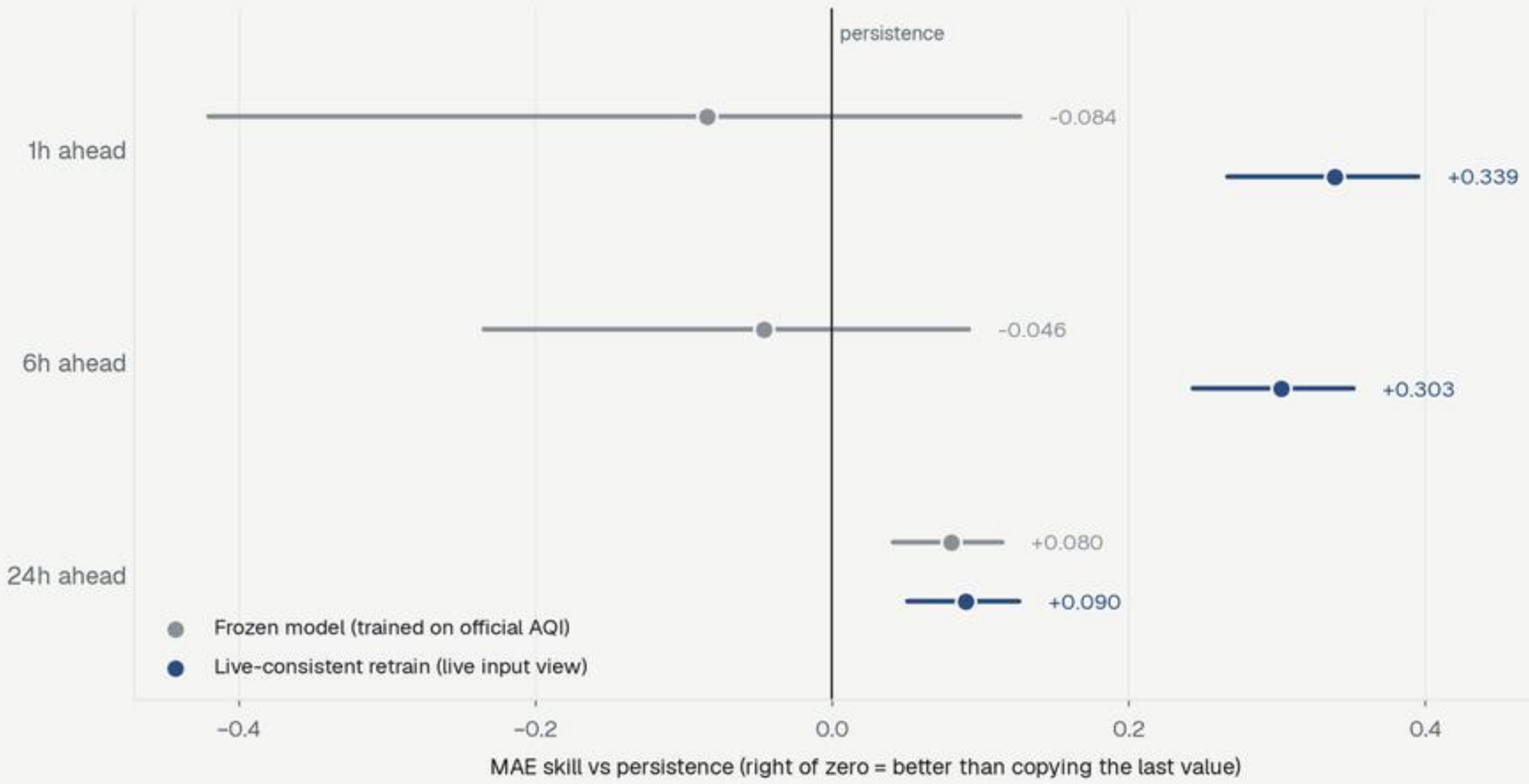
Source: .github/workflows/*.yml (schedules), docs/ARCHITECTURE.md, docs/decisions/

Ingest at :10, forecast at :25, scoring nightly at 20:40 UTC. One database; the API and dashboard only read.

Fixing train/serve skew: 6h skill −0.05 → +0.30

Train/serve skew: the frozen models do not carry over

Test winter, AQI recomputed from pollutants (what the live feed can compute). Skill vs persistence, 95% CI.



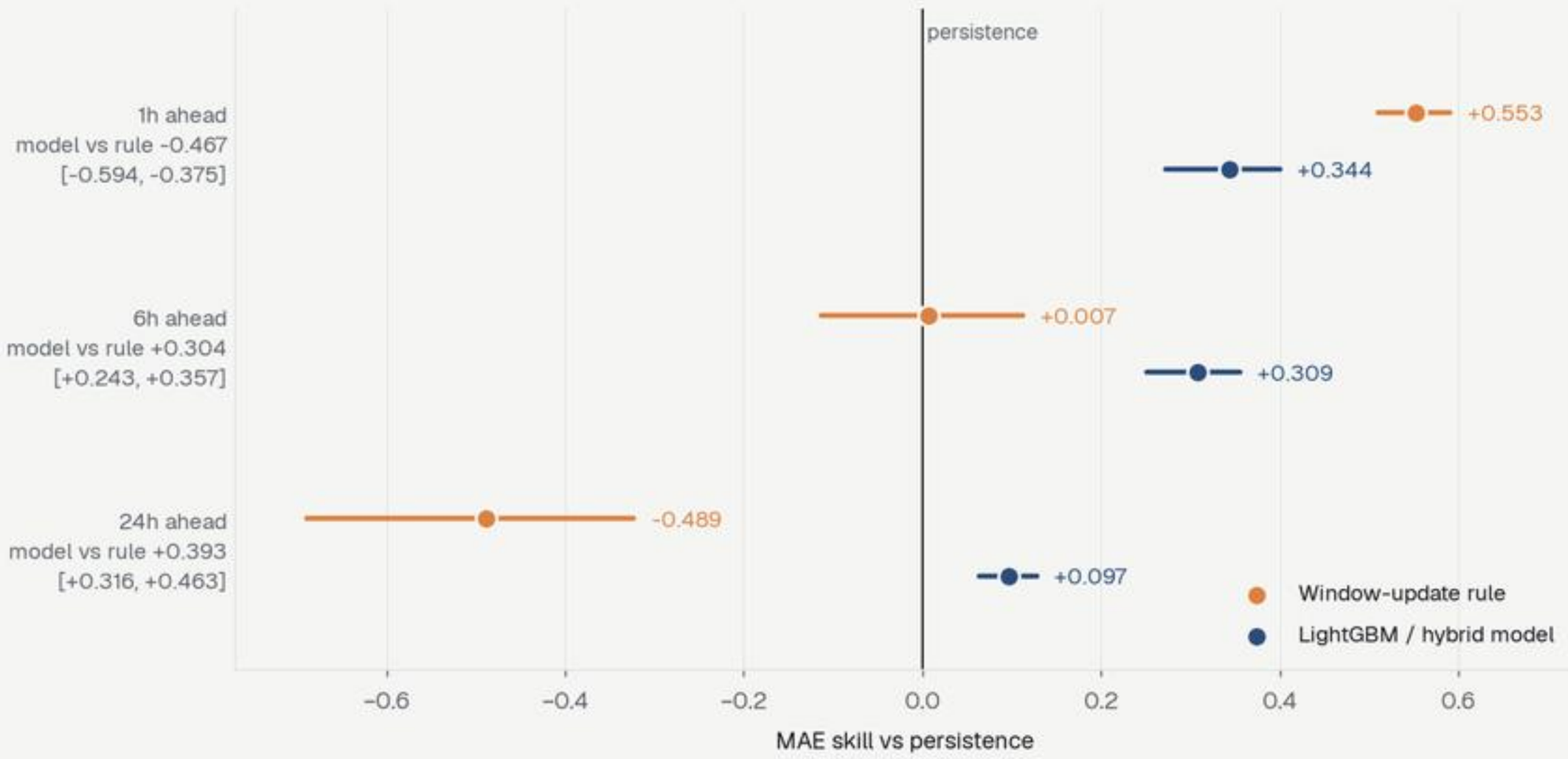
Source: reports/modeling/live_retrain/live_retrain_metrics.csv (test split, scored once)

Trained on official AQI, the frozen 6h model lost to persistence (–0.05). Retrained on what the live feed can compute: +0.30.

A simple rule wins at 1h, ML wins at 6h

A parameter-free rule wins at 1h; the model wins at 6h

Test winter, recomputed AQI. Skill vs plain persistence, 95% CI. The rule is window-update persistence (no fitted parameters).

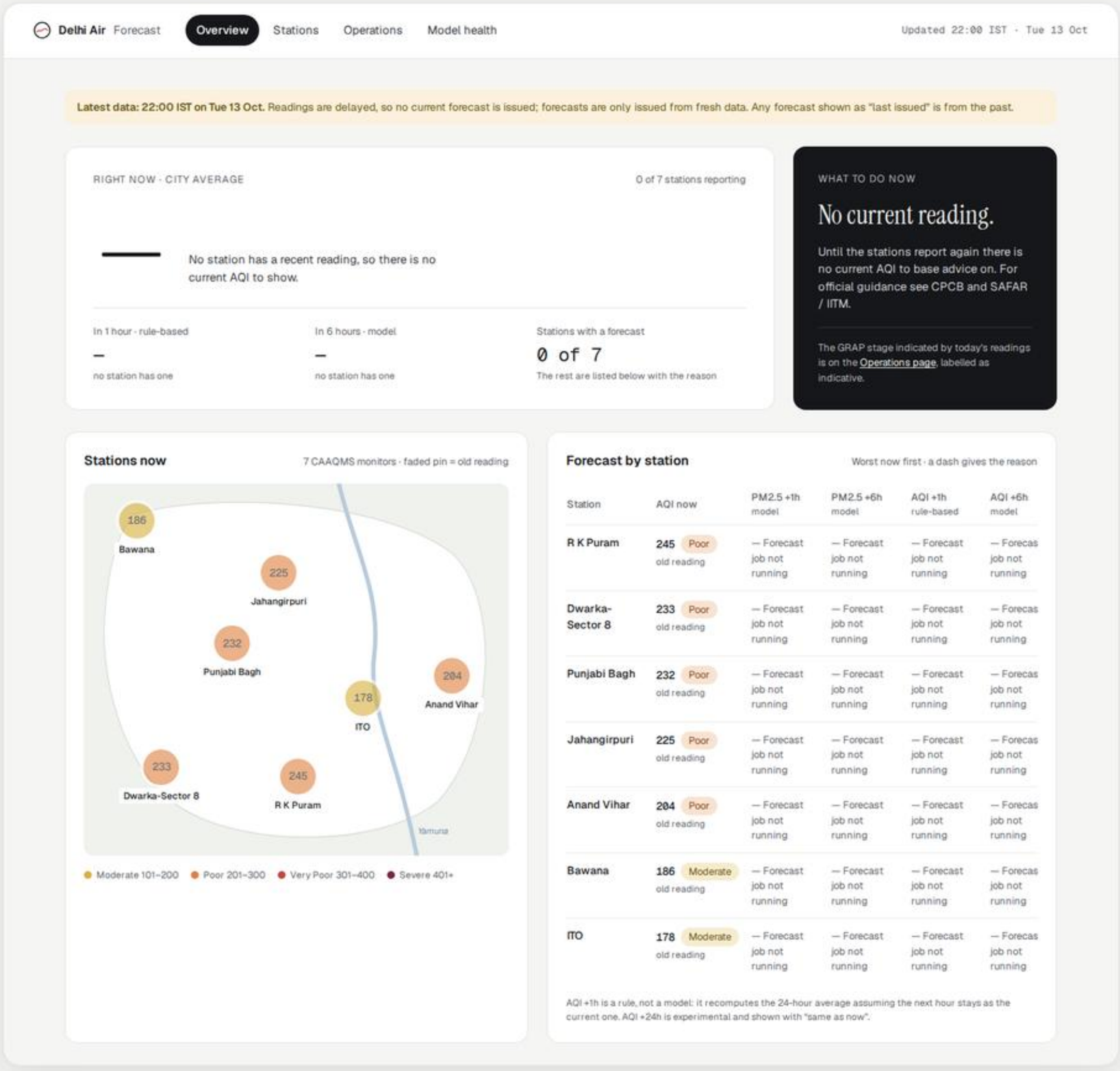


At 24h a flat-hour assumption for a whole day is a poor forecast, so the rule is not a fair comparator there; the 24h claim is the model against plain persistence.

Source: reports/modeling/live_retrain/window_update_baseline.csv (test split)

1h: the rule cuts error 55% vs persistence, the model 34%. 6h: the model 31%, the rule 1%.

Live dashboard



Code & live link in the comments